# Detección de Productos en Tiempo Real para Autocobro en Despensa Comunitaria

Visión por Computadora II — CEIA FIUBA

---

María Belén Cattaneo  Nicolás Valentín Ciarrapico  Sabrina Daiana Pryszczuk

Abril 2026

CEIA — Facultad de Ingeniería — Universidad de Buenos Aires

# El Problema

**Contexto**
Una **mutual** gestiona una despensa comunitaria con ∼60 productos.

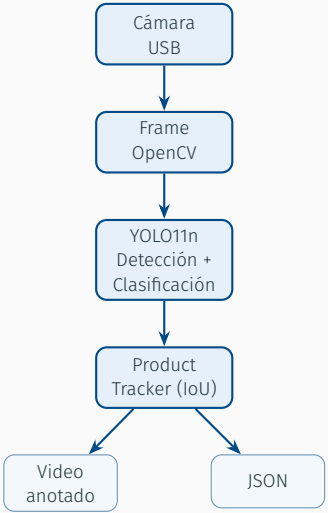
- El proceso de checkout es **manual** y lento
- Soluciones comerciales (Amazon Go) cuestan **millones de USD**
- Se necesita un sistema **accesible** para organizaciones pequeñas

**Nuestra propuesta**
Sistema de autocobro basado **exclusivamente en visión por computadora**, de bajo costo (**140–190 USD**).

# Solución Propuesta

1. Cámara cenital sobre mesa de control
2. El asociado coloca productos de a uno
3. El sistema detecta y clasifica en tiempo real
4. Se genera la lista de compra automáticamente

**Hardware**
Raspberry Pi 5 + webcam USB + monitor HDMI



# Dataset Propio

**POC: 10 de ∼60 clases**

1. aceite 1 l
2. aceite 4 l
3. dulce de leche
4. fideos
5. leche descremada
6. leche entera
7. miel
8. polenta
9. tomate
10. yerba

Incluye pares **visualmente similares**:
leche descremada/entera, aceite 1 l/4 l

**Composición**

| | Individual | Multi |
|---|---|---|
| Train | 102 | 132 |
| Validation | 13 | 19 |
| Test | 12 | 18 |
| **Total** | **127** | **169** |

Captura cenital, 1080p, fondo uniforme.
Anotado en **Roboflow** (formato YOLO).

# Modelo y Entrenamiento

## ¿Por qué YOLO?

- **Single-stage:** detección + clasificación en un paso
- **Tiempo real:** >15 FPS
- **Variante nano:** 2.5M parámetros
  $\Rightarrow$ viable en Raspberry Pi 5

## Configuración

| | |
|---|---|
| Épocas | 100 (early stop 20) |
| Batch size | 16 |
| Imagen | 640×640 |
| Optimizador | SGD (lr=0.01) |
| Transfer | COCO pre-trained |
| Hardware | Intel i5-6200U |

## Data Augmentation (crítico)

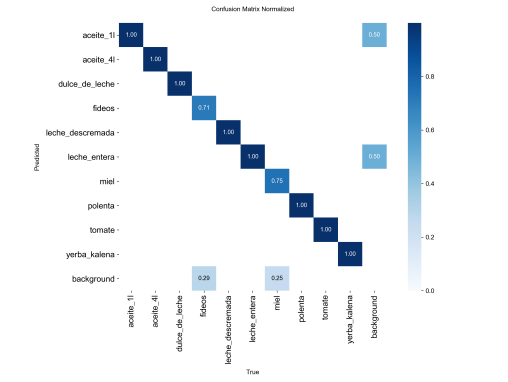
Mosaic ($p$=1.0) + flip horizontal ($p$=0.5) + jitter HSV. Sin augmentation: mAP@0.5 cae de 0.790 a 0.693 en test.

# Benchmark de Arquitecturas Nano

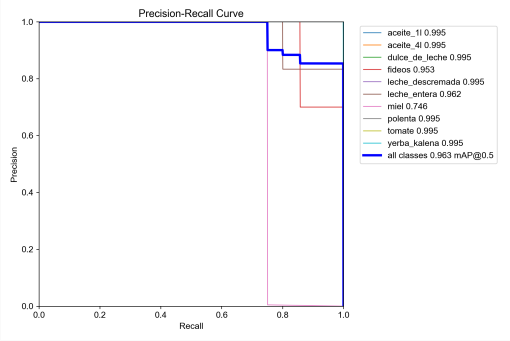
| Modelo | Parámetros | Inferencia | mAP@0.5 | mAP@0.5:0.95 |
|---|---|---|---|---|
| YOLOv8n | 3.0 M | 246.2 ms | **0.967** | 0.835 |
| YOLOv10n | **2.2 M** | 213.0 ms | 0.813 | 0.774 |
| YOLO11n | 2.5 M | **205.7 ms** | **0.967** | **0.889** |
| RT-DETR-L | — | — | *Segfault (sin memoria)* | |

**Conclusión**
**YOLO11n** seleccionado: mejor mAP@0.5:0.95 (**0.889**), mayor velocidad (**205.7 ms/img**), y empata en mAP@0.5 con YOLOv8n.

# Resultados: Validación Multi-Producto (YOLO11n)



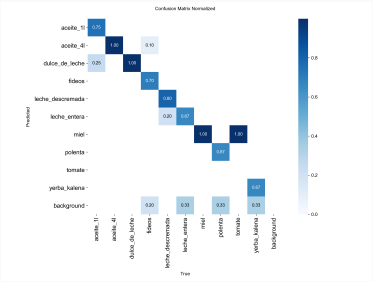

Matriz de confusión — validación



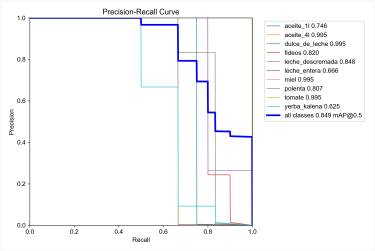

Curvas Precision-Recall — mAP@0.5 = **0.963**

9 de 10 clases con mAP@0.5 $> 0.95$, incluyendo pares similares (leche, aceite).
Clase problemática: *miel* (mAP@0.5 = 0.746).

# Resultados: Test Split Multi-Producto (YOLO11n)




Matriz de confusión — test




Curvas PR — mAP@0.5 = **0.849**

| Métrica | Valor |
|---|---|
| mAP@0.5 | 0.849 |
| mAP@0.5:0.95 | 0.733 |
| Precision | 0.948 |
| Recall | 0.709 |

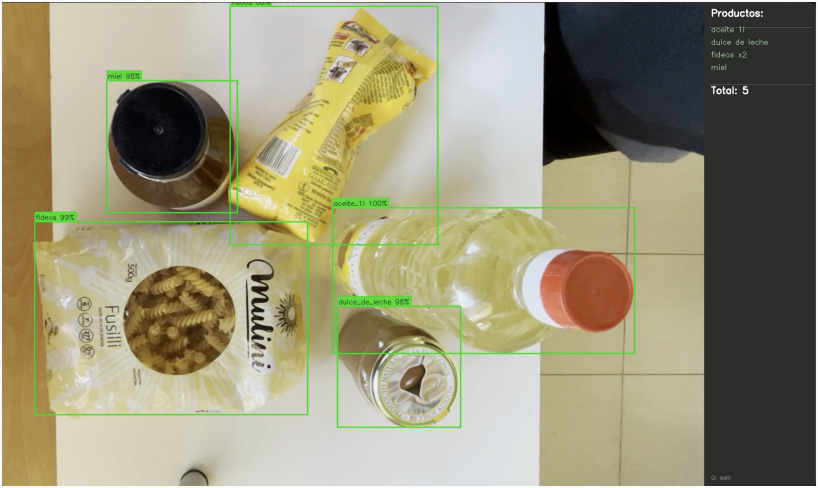
Perfectas: *aceite_4l*, *dulce_de_leche*, *miel*, *tomate* (mAP = 0.995)  Débiles: *yerba_kalena* (0.625), *leche_entera* (0.666)

# Predicciones del Modelo



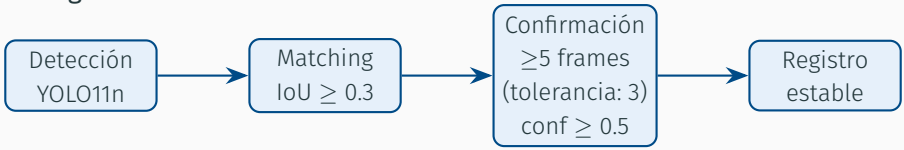

Detecciones con bounding boxes y nivel de confianza generadas por YOLO11n.

# Demo


miel 98%
fideos 99%
aceite_1l 100%
dulce_de_leche 98%
Fusilli
500g
Productos:
aceite 1l
dulce de leche
fideos x2
miel
Total: 5

# Pipeline de Inferencia

## Lógica de registro multi-frame




- **Verde:** producto confirmado
- **Amarillo:** pendiente de confirmación
- **Gris:** advertencia (baja confianza)

- Tolerancia de oclusión: 10 frames (∼333 ms)
- Confirmados suprimen duplicados y warnings en su región
- Salida: video anotado + JSON

# Conclusiones

1. Sistema de autocobro funcional basado **exclusivamente en CV**, costo **140–190 USD**
2. YOLO11n alcanza **mAP@0.5 = 0.963** en validación, **0.849** en test (multi-producto)
3. Inferencia en tiempo real: $>$15 FPS en laptop, $\sim$10 FPS en Raspberry Pi 5
4. Data augmentation **crítico**: sin él, mAP cae de 0.790 a 0.693
5. Lógica multi-frame elimina falsos positivos sin tracker complejo

## Trabajo Futuro

- Escalar a $\sim$60 clases del catálogo completo
- Deploy en Raspberry Pi 5 en condiciones reales
- Métricas end-to-end (order-level accuracy)
- Integración con sistema de gestión de la mutual

# ¡Gracias!

¿Preguntas?

**Repositorio:** github.com/BelCattaneo/vpc2_autocobro

Cattaneo · Ciarrapico · Pryszczuk
Grupo 4 — CEIA FIUBA — 2026